DESIGN AND ANALYSIS OF ALGORITHMS

DAA Practical Programs in C++

All 9 practicals with source code, sample output, time complexity, and space complexity.

Practical Reference Document

Complexity Summary

No.	Practical	Time Complexity	Space Complexity
1	Selection Sort	O(n^2)	O(1)
2	Merge Sort	O(n log n)	O(n)
3	Dijkstra's Algorithm	O(V^2)	O(V^2)
4	Travelling Salesman Problem	O(n!)	O(n)
5	Depth First Search (DFS)	O(V^2)	O(V)
6	N-Queen Problem (N = 4)	O(N!) (upper-bound estimate)	O(N)
7	TSP using Branch and Bound	O(n!) worst case	O(n)
8	Naive String Matching	O(nm)	O(1), excluding input strings
9	Min-Max Algorithm	O(b^d)	O(d)

Notation: n = number of items/cities, V = number of graph vertices, N = chessboard size, m = pattern length, b = branching factor, d = game-tree depth.
Notes: Dijkstra requires non-negative edge weights. The TSP examples use a fixed 4-city matrix. The N-Queen program prints one valid solution for N = 4.

1. Selection Sort

Aim: To sort an array using Selection Sort.

Time Complexity: O(n^2) **Space Complexity:** O(1)

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int a[] = {64, 25, 12, 22, 11};
    int n = 5, minIndex;

    for (int i = 0; i < n - 1; i++) {
        minIndex = i;

        for (int j = i + 1; j < n; j++) {
            if (a[j] < a[minIndex])
                minIndex = j;
        }

        swap(a[i], a[minIndex]);
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

Sample Output:

```
Sorted array: 11 12 22 25 64
```

2. Merge Sort

Aim: To sort an array using Merge Sort.

Time Complexity: O(n log n) **Space Complexity:** O(n)

Program:

```
#include <iostream>
using namespace std;

void merge(int a[], int l, int m, int r) {
    int b[100], i = l, j = m + 1, k = l;

    while (i <= m && j <= r) {
        if (a[i] <= a[j])
            b[k++] = a[i++];
        else
            b[k++] = a[j++];
    }

    while (i <= m) b[k++] = a[i++];
    while (j <= r) b[k++] = a[j++];

    for (i = l; i <= r; i++)
        a[i] = b[i];
}

void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

int main() {
    int a[] = {38, 27, 43, 3, 9, 82, 10};
    int n = 7;

    mergeSort(a, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

Sample Output:

```
Sorted array: 3 9 10 27 38 43 82
```

3. Dijkstra's Algorithm

Aim: To find shortest distances from a source vertex using Dijkstra's algorithm.

Time Complexity: O(V^2) **Space Complexity:** O(V^2)

Program:

```
#include <iostream>
using namespace std;

int main() {
    const int n = 5;
    const int INF = 9999;

    int g[n][n] = {
        {0, 10, 0, 0, 5},
        {0, 0, 1, 0, 2},
        {0, 0, 0, 4, 0},
        {7, 0, 6, 0, 0},
        {0, 3, 9, 2, 0}
    };

    int dist[n], visited[n] = {0};
    int source = 0;

    for (int i = 0; i < n; i++)
        dist[i] = INF;
    dist[source] = 0;

    for (int i = 0; i < n; i++) {
        int u = -1;

        for (int j = 0; j < n; j++) {
            if (!visited[j] && (u == -1 || dist[j] < dist[u]))
                u = j;
        }

        if (u == -1 || dist[u] == INF)
            break;

        visited[u] = 1;

        for (int v = 0; v < n; v++) {
            if (g[u][v] != 0 && !visited[v] &&
                dist[u] + g[u][v] < dist[v]) {
                dist[v] = dist[u] + g[u][v];
            }
        }
    }

    for (int i = 0; i < n; i++)
        cout << "Distance to " << i << " = " << dist[i] << endl;

    return 0;
}
```

Sample Output:

```
Distance to 0 = 0
Distance to 1 = 8
Distance to 2 = 9
Distance to 3 = 7
Distance to 4 = 5
```

4. Travelling Salesman Problem

Aim: To find the minimum-cost tour visiting each city once and returning to the starting city.

Time Complexity: O(n!) **Space Complexity:** O(n)

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

int main() {
    int cost[4][4] = {
        {0, 10, 15, 20},
        {10, 0, 35, 25},
        {15, 35, 0, 30},
        {20, 25, 30, 0}
    };

    int path[] = {1, 2, 3};
    int minCost = 999999;

    do {
        int total = cost[0][path[0]]
                  + cost[path[0]][path[1]]
                  + cost[path[1]][path[2]]
                  + cost[path[2]][0];

        if (total < minCost)
            minCost = total;
    } while (next_permutation(path, path + 3));

    cout << "Minimum cost: " << minCost;
    return 0;
}
```

Sample Output:

```
Minimum cost: 80
```

5. Depth First Search (DFS)

Aim: To traverse a graph using Depth First Search.

Time Complexity: O(V^2) **Space Complexity:** O(V)

Program:

```
#include <iostream>
using namespace std;

const int N = 5;

int graph[N][N] = {
    {0, 1, 1, 0, 0},
    {1, 0, 0, 1, 1},
    {1, 0, 0, 0, 1},
    {0, 1, 0, 0, 0},
    {0, 1, 1, 0, 0}
};

int visited[N] = {0};

void DFS(int v) {
    cout << v << " ";
    visited[v] = 1;

    for (int i = 0; i < N; i++) {
        if (graph[v][i] == 1 && !visited[i])
            DFS(i);
    }
}

int main() {
    cout << "DFS Traversal: ";
    DFS(0);
    return 0;
}
```

Sample Output:

```
DFS Traversal: 0 1 3 4 2
```

6. N-Queen Problem (N = 4)

Aim: To place four queens on a chessboard so that no two queens attack each other.

Time Complexity: O(N!) (upper-bound estimate) **Space Complexity:** O(N)

Program:

```
#include <iostream>
#include <cmath>
using namespace std;

const int N = 4;
int queen[N];

bool safe(int row, int col) {
    for (int i = 0; i < row; i++) {
        if (queen[i] == col ||
            abs(queen[i] - col) == abs(i - row))
            return false;
    }
    return true;
}

bool solve(int row) {
    if (row == N)
        return true;

    for (int col = 0; col < N; col++) {
        if (safe(row, col)) {
            queen[row] = col;
            if (solve(row + 1))
                return true;
        }
    }
    return false;
}

int main() {
    if (solve(0)) {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++)
                cout << (queen[i] == j ? "Q " : ". ");
            cout << endl;
        }
    } else {
        cout << "No solution found.";
    }

    return 0;
}
```

Sample Output:

```
. Q . .
. . . Q
Q . . .
. . Q .
```

7. TSP using Branch and Bound

Aim: To solve the Travelling Salesman Problem using Branch and Bound pruning.

Time Complexity: O(n!) worst case **Space Complexity:** O(n)

Program:

```
#include <iostream>
#include <climits>
#include <algorithm>
using namespace std;

const int N = 4;

int cost[N][N] = {
    {0, 10, 15, 20},
    {10, 0, 35, 25},
    {15, 35, 0, 30},
    {20, 25, 30, 0}
};

int best = INT_MAX;

void tsp(int city, int count, int totalCost, int visited) {
    // All edge costs are non-negative, so this safely prunes
    // paths that already cost at least as much as the best tour.
    if (totalCost >= best)
        return;

    if (count == N) {
        best = min(best, totalCost + cost[city][0]);
        return;
    }

    for (int next = 1; next < N; next++) {
        if (!(visited & (1 << next))) {
            tsp(next, count + 1, totalCost + cost[city][next],
                visited | (1 << next));
        }
    }
}

int main() {
    tsp(0, 1, 0, 1);
    cout << "Minimum cost: " << best;
    return 0;
}
```

Sample Output:

```
Minimum cost: 80
```

8. Naive String Matching

Aim: To find occurrences of a pattern in a text using Naive String Matching.

Time Complexity: O(nm) **Space Complexity:** O(1), excluding input strings

Program:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "AABAACAADAABAABA";
    string pattern = "AABA";

    int n = text.length();
    int m = pattern.length();

    for (int i = 0; i <= n - m; i++) {
        int j;

        for (j = 0; j < m; j++) {
            if (text[i + j] != pattern[j])
                break;
        }

        if (j == m)
            cout << "Pattern found at index " << i << endl;
    }

    return 0;
}
```

Sample Output:

```
Pattern found at index 0
Pattern found at index 9
Pattern found at index 12
```

9. Min-Max Algorithm

Aim: To demonstrate the Min-Max algorithm on a small game tree.

Time Complexity: O(b^d) **Space Complexity:** O(d)

Program:

```
#include <iostream>
#include <algorithm>
using namespace std;

// A full binary game tree with 4 leaf values (depth = 2).
int minimax(int depth, int node, bool isMax, int values[]) {
    if (depth == 2)
        return values[node];

    if (isMax) {
        return max(
            minimax(depth + 1, node * 2, false, values),
            minimax(depth + 1, node * 2 + 1, false, values)
        );
    }

    return min(
        minimax(depth + 1, node * 2, true, values),
        minimax(depth + 1, node * 2 + 1, true, values)
    );
}

int main() {
    int values[] = {3, 5, 2, 9};

    cout << "Best value: " << minimax(0, 0, true, values);
    return 0;
}
```

Sample Output:

```
Best value: 3
```